VERİ ANALİSTİ MÜFREDATI 2026

MODÜL 9

PYTHON

Python Temeli • NumPy • Pandas • DataFrame • Filtering • GroupBy • Merge • Missing Values • Datetime

Modül çıktısı
Bir veri analisti gibi Python ile veriyi yükleyebilir, inceleyebilir, temizleyebilir, filtreleyebilir, özetleyebilir, tabloları birleştirebilir, tarih verisiyle çalışabilir ve tekrar üretilebilir bir EDA akışı kurabilirsin.

1. Veri Analisti Neden Python Kullanır?

Excel hızlı keşif ve küçük veri setleri için çok güçlüdür; SQL veriyi kaynaktan çekmek için temeldir. Python ise tekrarlanabilir veri temizleme, otomasyon, büyük veri setleri üzerinde dönüşüm, istatistiksel analiz ve EDA süreçlerini tek akışta birleştirir.

Araç	En güçlü olduğu alan	Python ile ilişki
Excel	Hızlı manuel analiz, pivot, iş kullanıcıları	Python tekrarlayan işleri otomatikleştirir.
SQL	Veritabanından doğru veriyi çekmek	Python SQL çıktısını analiz eder ve dönüştürür.
Python	Temizleme, EDA, otomasyon, istatistik	Analiz pipeline'ının programlanabilir katmanıdır.
Power BI	Dashboard ve paylaşım	Python ön işleme / keşif için kullanılabilir.

Analist bakış açısı
Amaç "Python kodu yazmak" değil, iş sorusunu güvenilir ve tekrar üretilebilir şekilde yanıtlamaktır. Her kod satırının veri veya iş açısından bir nedeni olmalıdır.

2. Python Temelleri

2.1 Değişkenler ve temel veri tipleri

```
revenue = 125000.50
orders = 1840
country = "Turkey"
is_active = True
```

Tip	Örnek	Analitik kullanım

int	orders = 1840	Sipariş adedi, kullanıcı sayısı
float	revenue = 125000.50	Gelir, oran, ortalama
str	country = "Turkey"	Kategori, ID, metin
bool	is_active = True	Filtre ve koşul mantığı
None	discount = None	Eksik / bilinmeyen değer

2.2 List, tuple, dictionary, set

```
cities = ["Istanbul", "Ankara", "Izmir"]
metrics = ("revenue", "orders")
order = {"order_id": 101, "revenue": 450.0}
unique_cities = {"Istanbul", "Ankara"}
```

Veri analistliği için bu yapıların mantığını bilmek yeterlidir; asıl yoğun kullanım Pandas DataFrame üzerinde olur.

2.3 Koşullar ve döngüler

```
if revenue > 100000:
    status = "High"
else:
    status = "Normal"


for city in cities:
    print(city)
```

Ne kadar derin öğrenmelisin?
Data analyst rolünde ileri OOP, design patterns veya algoritma yarışması düzeyi Python öncelikli değildir. Fonksiyonlar, veri yapıları, hata okuma ve kütüphane kullanımı çok daha değerlidir.

3. Fonksiyonlar, Import ve Notebook Mantığı

3.1 Fonksiyon yazmak

```
def conversion_rate(orders, sessions):
    if sessions == 0:
        return 0
    return orders / sessions
```

Fonksiyonlar tekrar eden analitik mantığı tek yerde tutar. Böylece hesaplama standardı korunur.

3.2 Import

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

3.3 Jupyter Notebook çalışma biçimi

- Hücreleri mantıksal bloklara ayır: import, load, validation, cleaning, EDA, insights.
- Bir hücrenin yalnızca daha önce manuel çalıştırılmış gizli state'e bağlı olmamasına dikkat et.

- Notebook baştan sona “Run All” ile çalışabilmelidir.

4. NumPy Temelleri

NumPy sayısal hesaplamanın temelidir. Pandas da büyük ölçüde NumPy ekosistemi üzerinde çalışır. Veri analisti için array mantığı, vektörize işlem, temel istatistik ve NaN davranışları önemlidir.

```
import numpy as np


values = np.array([120, 150, 90, 210])
print(values.mean())
print(values.std())
print(values.max())
```

4.1 Vektörize işlem

```
prices = np.array([100, 200, 300])
discounted = prices * 0.90
```

Python listesi üzerinde tek tek döngü kurmak yerine array üzerinde toplu işlem yapmak genellikle daha kısa ve daha hızlıdır.

4.2 np.where

```
segments = np.where(values >= 150, "High", "Low")
```

5. Pandas ve DataFrame Mantığı

Pandas veri analistinin Python’daki ana çalışma aracıdır. DataFrame satır ve sütunlardan oluşan tablo benzeri bir yapıdır; ancak Excel’den farklı olarak işlemler kodla tanımlanır ve tekrarlanabilir.

```
import pandas as pd


df = pd.read_csv("orders.csv")
print(df.head())
print(df.shape)
print(df.info())
```

Komut	Ne yapar?	Neden önemli?
df.head()	İlk satırları gösterir	Hızlı içerik kontrolü
df.shape	Satır ve sütun sayısı	Grain / hacim kontrolü
df.columns	Sütun adları	Şema kontrolü
df.dtypes	Veri tipleri	Yanlış tipleri bulma
df.info()	Tip + null özeti	İlk kalite taraması
df.describe()	Sayısal özet	Dağılım ve outlier sinyali

6. Veri Yükleme ve İlk Profiling

6.1 CSV ve Excel

```
df = pd.read_csv("orders.csv")
products = pd.read_excel("products.xlsx", sheet_name="Products")
```

6.2 İlk 60 saniyelik kontrol

```
df.shape
df.head()
df.dtypes
df.isna().sum()
df.duplicated().sum()
df.nunique()
```

Altın kural
Analize başlamadan önce verinin grain'ini tanımla: Bir satır neyi temsil ediyor? Sipariş mi, sipariş kalemi mi, müşteri mi, event mi? Grain yanlış anlaşılırsa metrikler yanlış çıkar.

7. Sütun ve Satır Seçimi

7.1 Sütun seçmek

```
df["revenue"]
df[["order_id", "customer_id", "revenue"]]
```

7.2 loc ve iloc

```
df.loc[df["revenue"] > 1000, ["order_id", "revenue"]]
df.iloc[0:5, 0:3]
```

loc etiket/sütun adı mantığıyla, iloc pozisyon mantığıyla çalışır. Analizde çoğunlukla loc daha okunaklıdır.

8. Filtering - İş Sorularını Satır Filtrelerine Çevirmek

```
high_value = df[df["revenue"] > 1000]


istanbul = df[df["city"] == "Istanbul"]


filtered = df[(df["revenue"] > 1000) & (df["status"] == "Completed")]
```

8.1 isin, between ve string filtreleri

```
df[df["city"].isin(["Istanbul", "Ankara"])]
df[df["revenue"].between(500, 1500)]
df[df["product_name"].str.contains("phone", case=False, na=False)]
```

Yaygın hata
Birden fazla Pandas koşulunda and/or yerine & ve | kullanılır ve her koşul paranteze alınır.

9. Yeni Sütun Üretmek

```
df["profit"] = df["revenue"] - df["cost"]
df["margin"] = df["profit"] / df["revenue"]
```

9.1 Koşullu segment

```
df["value_segment"] = np.where(
    df["revenue"] >= 1000,
    "High Value",
    "Standard"
)
```

9.2 assign

```
df = df.assign(
    profit=df["revenue"] - df["cost"],
    margin=lambda x: x["profit"] / x["revenue"]
)
```

10. Missing Values

Eksik değer gördüğünde ilk soru “Nasıl doldururum?” değil, “Neden eksik?” olmalıdır. Eksik veri; gerçek yokluk, sistem hatası, join sorunu veya iş sürecinin doğal sonucu olabilir.

```
df.isna().sum()
df.isna().mean().sort_values(ascending=False)
```

10.1 dropna ve fillna

```
df_clean = df.dropna(subset=["order_id"])
df["discount"] = df["discount"].fillna(0)
df["city"] = df["city"].fillna("Unknown")
```

Durum	Olası yaklaşım	Risk
Kimlik alanı eksik	Satırı incele / gerekirse dışla	Grain bozulabilir
İndirim eksik	İş kuralı 0 ise 0 doldur	Bilinmeyen ile 0 karışabilir
Kategori eksik	Unknown kategorisi	Raporlamada açıkça göster
Sayısal ölçüm eksik	Medyan/ortalama sadece gerekçeli ise	Dağılımı yapay değiştirebilir

11. Duplicate Kayıtlar

```
df.duplicated().sum()
df[df.duplicated(keep=False)]


df.duplicated(subset=["order_id"]).sum()
```

Tüm satır duplicate olması ile business key duplicate olması aynı şey değildir. Örneğin order_items tablosunda order_id'nin tekrar etmesi normaldir; çünkü bir sipariş birden fazla ürüne sahip olabilir.

Kontrol sorusu
Duplicate kararını her zaman tablonun grain'ine göre ver. "Tekrarlı görünüyor" diye satır silmek veri kaybına yol açabilir.

12. GroupBy ve Aggregation

groupby, SQL'deki GROUP BY mantığının Pandas karşılığıdır. Analistin en sık kullandığı işlemlerden biridir.

```
city_summary = (
    df.groupby("city")
      .agg(
          revenue=("revenue", "sum"),
          orders=("order_id", "nunique"),
          avg_order_value=("revenue", "mean")
      )
      .reset_index()
)
```

12.1 Birden fazla kırılım

```
summary = (
    df.groupby(["month", "category"])
      .agg(revenue=("revenue", "sum"), orders=("order_id", "nunique"))
      .reset_index()
)
```

12.2 transform

```
df["category_avg"] = df.groupby("category")["revenue"].transform("mean")
df["vs_category_avg"] = df["revenue"] / df["category_avg"]
```

13. Sort, Rank ve Top-N

```
top_products = (
    df.groupby("product_name", as_index=False)["revenue"].sum()
      .sort_values("revenue", ascending=False)
      .head(10)
)
df["revenue_rank"] = df["revenue"].rank(method="dense", ascending=False)
```

14. Merge - Tabloları Birleştirmek

Pandas merge SQL JOIN mantığıyla çalışır. En kritik konu join key ve cardinality'dir.

```
analysis = orders.merge(
    customers,
    on="customer_id",
    how="left",
    validate="many_to_one"
)
```

how	SQL karşılığı	Kullanım
inner	INNER JOIN	Sadece iki tabloda eşleşenler
left	LEFT JOIN	Ana tabloyu koru
right	RIGHT JOIN	Sağ tabloyu koru
outer	FULL OUTER JOIN	Tüm kayıtları gör

14.1 Merge sonrası mutlaka kontrol et

```
before = len(orders)
analysis = orders.merge(customers, on="customer_id", how="left")
after = len(analysis)


print(before, after)
print(analysis["customer_id"].isna().sum())
```

Kritik analist hatası
Many-to-many merge satır sayısını patlatabilir ve revenue gibi ölçüleri double-count edebilir. Merge öncesi key uniqueness, merge sonrası satır sayısı ve toplam metrik kontrolü yap.

15. Concat

```
all_orders = pd.concat([orders_2025, orders_2026], ignore_index=True)
```

concat çoğunlukla aynı şemadaki tabloları alt alta eklemek için kullanılır. SQL'de UNION ALL'a benzer.

16. Datetime ile Çalışmak

```
df["order_date"] = pd.to_datetime(df["order_date"], errors="coerce")
df["year"] = df["order_date"].dt.year
df["month"] = df["order_date"].dt.to_period("M")
df["weekday"] = df["order_date"].dt.day_name()
```

16.1 Tarih filtresi

```
start = pd.Timestamp("2026-01-01")
end = pd.Timestamp("2026-03-31")
q1 = df[df["order_date"].between(start, end)]
```

16.2 Süre hesaplama

```
df["delivery_days"] = (df["delivery_date"] - df["order_date"]).dt.days
```

Zaman analizi notu
Aylık karşılaştırmalarda eksik gün, farklı ay uzunluğu, timezone, iptal/iade tarihleri ve partial-period etkisini kontrol et.

17. Kategorik ve Metin Verisi Temizleme

```
df["city"] = (
    df["city"]
      .str.strip()
      .str.title()
)


df["email_domain"] = df["email"].str.split("@").str[-1]
```

17.1 replace ve map

```
df["channel"] = df["channel"].replace({
    "fb": "Facebook",
    "ig": "Instagram"
})


segment_map = {1: "Low", 2: "Medium", 3: "High"}
df["segment_name"] = df["segment_id"].map(segment_map)
```

18. EDA - Exploratory Data Analysis Akışı

EDA'nın amacı grafik üretmek değil, verinin yapısını ve iş davranışını anlamaktır.

1. İş sorusunu ve ana KPI'ları yaz.
2. Grain ve şemayı doğrula.
3. Missing, duplicate, invalid category ve tip kontrollerini yap.
4. Temel dağılımları ve uç değerleri incele.
5. KPI'yı zaman, kanal, ülke, kategori veya segment kırılımlarına ayır.
6. Beklenmeyen değişimin kaynağını hipotezlerle araştır.
7. Bulguyu iş etkisine çevir.

```
df["revenue"].describe()
df["channel"].value_counts(normalize=True)
df.groupby("category")["revenue"].sum().sort_values(ascending=False)
```

19. Uçtan Uca E-Ticaret Analizi

İş problemi

Yönetim son iki ayda gelirin düştüğünü söylüyor. Amaç düşüşün sipariş sayısı, müşteri davranışı, kategori, kanal veya ortalama sipariş değeri kaynaklı olup olmadığını bulmak.

Adım 1 - Veriyi yükle

```
orders = pd.read_csv("orders.csv")
customers = pd.read_csv("customers.csv")
```

Adım 2 - Kalite kontrolü

```
orders.shape
orders.dtypes
orders.isna().sum()
orders.duplicated(subset=["order_id"]).sum()
```

Adım 3 - Tarihi düzelt ve KPI üret

```
orders["order_date"] = pd.to_datetime(orders["order_date"])
orders["month"] = orders["order_date"].dt.to_period("M")
orders["profit"] = orders["revenue"] - orders["cost"]
```

Adım 4 - Aylık özet

```
monthly = (
    orders.groupby("month")
     .agg(
        revenue=("revenue", "sum"),
        orders=("order_id", "nunique"),
        customers=("customer_id", "nunique")
     )
     .reset_index()
)
monthly["aov"] = monthly["revenue"] / monthly["orders"]
```

Adım 5 - Root-cause kırılımı

```
by_channel = (
    orders.groupby(["month", "channel"], as_index=False)
        .agg(revenue=("revenue", "sum"), orders=("order_id", "nunique"))
)

by_category = (
    orders.groupby(["month", "category"], as_index=False)
        .agg(revenue=("revenue", "sum"), orders=("order_id", "nunique"))
)
```

Adım 6 - Insight cümlesi

Örnek çıktı
"Toplam gelir %12 gerilerken sipariş sayısı yalnızca %3 düştü. Ana etki AOV'daki düşüşten geliyor. AOV kaybının büyük kısmı Electronics kategorisinde ve Paid Social kanalında yoğunlaşıyor. Bir sonraki adım ürün karması, indirim oranı ve trafik kalitesini incelemektir."

20. Kod Kalitesi ve Analiz Güvenilirliği

- Magic number yerine anlamlı değişken kullan: MIN_REVENUE = 1000.
- Tekrar eden kodu fonksiyona dönüştür.
- Ham veriyi mümkünse overwrite etme; temiz veri için yeni DataFrame oluştur.
- Her merge ve aggregation sonrası row count / metric validation yap.
- Notebook sonunda ana KPI'ları tek bir validation hücresinde tekrar kontrol et.
- Dosya yollarını ve kolon isimlerini tutarlı kullan.

20.1 Basit assertion

```
assert orders["order_id"].notna().all()
assert orders["revenue"].ge(0).all()
assert orders["order_id"].is_unique
```

Assertion, varsayım bozulduğunda analizi sessizce yanlış sonuç üretmek yerine durdurur.

21. Hata Okuma ve Debug

Hata	Genelde ne demek?	İlk kontrol
KeyError	Sütun/anahtar bulunamadı	df.columns ve yazım
TypeError	Uyumsuz veri tipi	dtypes ve dönüşüm
ValueError	Geçersiz değer/shape	input yapısı
FileNotFoundError	Dosya yolu yanlış	path / çalışma klasörü
MergeError	validate kuralı bozuldu	key uniqueness

Debug yaklaşımı
Hata mesajının en altındaki exception satırını oku. Sonra hatayı üreten satırdaki objenin tipini, kolon adını, shape'ini ve birkaç örneğini kontrol et.

22. Veri Analisti İçin Python Mülakat Soruları

8. DataFrame ile Series arasındaki fark nedir?
9. loc ve iloc arasındaki farkı anlat.
10. Pandas'ta missing values nasıl tespit edilir? Her eksik değeri doldurmalı mıyız?
11. groupby + agg ile birden fazla KPI nasıl hesaplanır?
12. merge sonrası satır sayısı neden artabilir?
13. inner join ile left join arasındaki fark nedir?
14. apply neden her zaman ilk tercih olmamalıdır? Vektörize yaklaşım neden önemlidir?

15. Bir order_id duplicate görünüyorsa bunu doğrudan siler misin? Neden?
16. Bir tarih kolonu object geliyorsa ne yaparsın?
17. Bir analizin tekrar üretilebilir olduğunu nasıl garanti edersin?
18. Aylık revenue düşüşünü Python ile hangi sıra ile araştırırsın?
19. Pandas çıktını SQL sonucu ile nasıl doğrularsın?

23. Uygulama Alıştırmaları

20. orders.csv dosyasını oku; shape, dtypes, null ve duplicate profilini çıkar.
21. Sadece Completed siparişleri filtrele.
22. Revenue > 1000 ve kanal Mobile olan siparişleri getir.
23. profit ve margin sütunlarını üret.
24. Şehir bazında revenue, unique order ve AOV hesapla.
25. Kategori bazında en yüksek gelirli ilk 5 kategoriyi bul.
26. customers tablosunu orders ile left merge et ve satır sayısını doğrula.
27. Müşteri bazında lifetime revenue hesapla.
28. order_date kolonundan month ve weekday üret.
29. Aylık revenue büyümesini hesapla.
30. Null discount değerlerini iş kuralına göre işle ve kararını açıklayan not yaz.
31. Analizin sonunda 3 insight + 2 recommendation üret.

24. Portfolyo Projesi - Python E-Commerce EDA

GitHub'a koyacağın proje yalnızca notebook olmamalı; iş problemi ve karar çıktısı içermelidir.

Bölüm	Beklenen çıktı
Business Context	Şirket problemi ve sorular
Dataset	Tablolar, kolonlar, grain
Data Quality	Missing, duplicate, type ve rule kontrolleri
Cleaning	Tekrar üretilebilir Pandas pipeline
EDA	Trend, segment, kategori, kanal analizleri
Root Cause	Ana KPI değişiminin sürücüleri
Insights	Verinin söylediği 3-5 önemli bulgu
Recommendations	Aksiyon + beklenen iş etkisi
README	Kurulum, yöntem, sonuç, görseller

CV cümlesi örneği
"Built an end-to-end e-commerce EDA workflow in Python/Pandas, validating data quality, merging customer and order data, analyzing revenue drivers, and translating findings into business recommendations."

25. Hızlı Referans - Cheat Sheet

İhtiyaç	Pandas
Dosya oku	pd.read_csv(...)
İlk satırlar	df.head()
Boyut	df.shape

Tipler	df.dtypes
Null	df.isna().sum()
Duplicate	df.duplicated().sum()
Filtre	df[df["x"] > 0]
Sıralama	df.sort_values("x")
Grupla	df.groupby("x").agg(...)
Birleştir	df.merge(other, on="id", how="left")
Alt alta ekle	pd.concat([...])
Tarih dönüştür	pd.to_datetime(df["date"])
Unique adet	df["id"].nunique()
Frekans	df["category"].value_counts()
Yeni sütun	df["profit"] = df["rev"] - df["cost"]

26. Modül Sonu Kontrol Listesi

- ☐ Python temel veri tiplerini ve koşul/fonksiyon mantığını okuyabiliyorum.
- ☐ NumPy array ve vektörize işlem mantığını biliyorum.
- ☐ Pandas ile CSV/Excel yükleyip DataFrame'i profilleyebiliyorum.
- ☐ Filtreleme, sütun üretme ve veri tipi dönüşümü yapabiliyorum.
- ☐ Missing values ve duplicate kayıtları iş bağlamına göre ele alabiliyorum.
- ☐ groupby/agg ile business KPI üretebiliyorum.
- ☐ merge yaparken cardinality ve double-count riskini kontrol ediyorum.
- ☐ datetime alanlarından dönemsel analiz üretebiliyorum.
- ☐ Bir EDA sürecini baştan sona notebook içinde kurabiliyorum.
- ☐ Koddan çıkan sonucu insight ve recommendation'a çevirebiliyorum.

Sonraki modül
Modül 10 - Görselleştirme: Matplotlib, doğru chart seçimi, kötü grafiklerden kaçınma, data storytelling ve dashboard prensipleri.